# *STARK*

## A learning group for ZK and SNARK application development

***Daniel Szego***

In: https://www.linkedin.com/in/daniel-szego/

# Agenda

- *Logistics*
- *Zero knowledge proofs - summary*
- *SNARK vs STARK*
- *Algorithm comparison*
- *Trusted setup*
- *High level flow of zero knowledge programming*
- *Arithmetic circuit*
- *Arithmetic Intermediate Representation (AIR*
- *Constructions*
- *Zoo of Zero Knowledge Proofs*
- *Applications*
- *Links, Resources, Q&A*

License: Apache 2.0

# Logistics

Every month or two months

Similar structure, similar logistics + ***guest lectures***

**Theory:**

- Look tables,
- Zk variations: STARKs, FRI
- recursive SNARKs

**Programming / platforms:**

- Zinc / ZKSync
- Cairo

**Applications:**

- ZK bridges
- ZK Virtual Machines
- ZK Identity / verifiable credential - presentations
- Voting

**Discord channel:**

*https://discord.com/channels/905194001349627914/1329201532628898036*

**Meetup.com:**

*https://www.meetup.com/lfdt-hungary/events/305634614/*

**Repo with all the contents:**

*https://github.com/LF-Decentralized-Trust-labs/zk-learning-group*

 License: Apache 2.0

# Zero knowledge proofs - summary

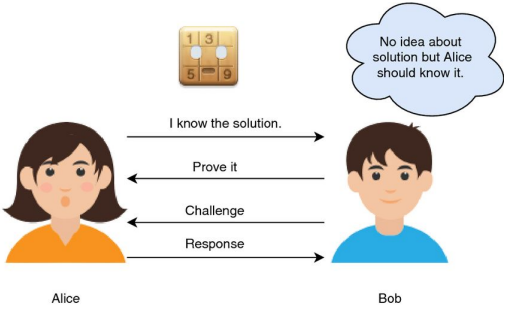
***Computation***: arithmetic circuit : *C( x, w ) -> F*
- *x* public input
- *w* private input, witness

***Prover*** algorithm: *P (x, w) -> proof*

***Verifier*** algorithm: *V (x, proof) -> accept / reject*

***Properties:***

- *Succinct:*
- *Complete:*
- *Knowledge sound:*
- *Zero knowledge*



License: Apache 2.0

# SNARK vs STARK

**SNARK**

- **Succinct:**

  Small in size and fast to verify

  However, proof generation might be resource intensive

- **Non-interactive:**

  No interaction between the prover and the verifier, the prover publishes the proof and the verifier either accepts or rejects

- **Argument of Knowledge:**

  The prover knows a fact to “prove”

Trusted setup: might be a workaround generating in a multiparty way with power of tau

**STARK**

- **Scalable:**

  Generating proof is fast (quasilinear, (O(nlog {2}n)))

  Verification time is polylogarithmic

  Different computational model

- **Transparent:**

  No trusted setup

- **Argument of knowledge:**

  The prover knows a fact to “prove”

It might be interactive or non-interactive

Fiat-Shamir helps transforming some interactive proofs to non-interactives.

License: Apache 2.0

# Algorithm comparison

Under very active development

Proof size

Verification time

Proving time matters as well

Setup :

- per circuit
- universal
- transparent

Post quantum readiness

| | size of proof $\pi$ | verifier time | setup | post-quantum? |
|---|---|---|---|---|
| Groth'16 | $\approx$ 200 Bytes $O_\lambda(1)$ | $\approx$ 1.5 ms $O_\lambda(1)$ | trusted per circuit | no |
| Plonk / Marlin | $\approx$ 400 Bytes $O_\lambda(1)$ | $\approx$ 3 ms $O_\lambda(1)$ | universal trusted setup | no |
| Bulletproofs | $\approx$ 1.5 KB $O_\lambda(\log \lvert C \rvert)$ | $\approx$ 3 sec $O_\lambda(\lvert C \rvert)$ | transparent | no |
| STARK | $\approx$ 100 KB $O_\lambda(\log^2 \lvert C \rvert)$ | $\approx$ 10 ms $O_\lambda(\log^2 \lvert C \rvert)$ | transparent | yes |

using "Replace Image" to show your own photo.

 License: Apache 2.0

# Trusted setup / power of tau

Compute compiled circuit representation, proving and verification keys:

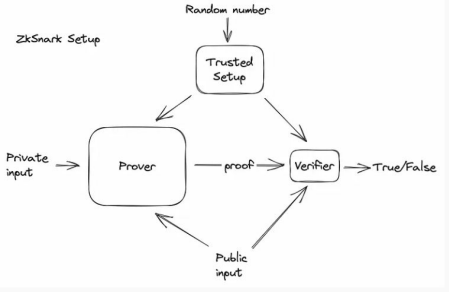
- Cheat / forge proofs
- Power of tau

Trusted setup:

- per circuit / universal /transparent

**Power of tau:**

- a multi-party computation (MPC) ceremony to generate parameters for zk-SNARKs:
- multiple participants contributing randomness
- one participant acts honestly, the resulting parameters remain secure and confidential
- "tau" refers to a secret value or trapdoor used in the parameter generation process.



License: Apache 2.0

# High level flow of zero knowledge programming

**Computation:**

- “High” level programs
- Input-output, program instructions

**Arithmetization:**

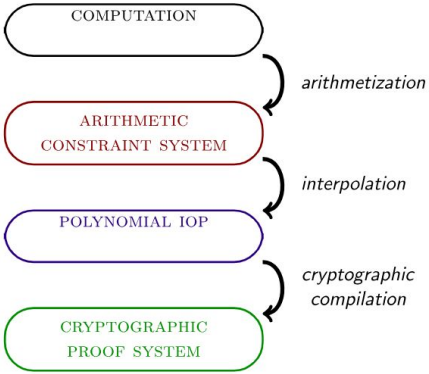
- Arithmetic constraint system
- Equations on finite fields
- Constraint system solution: valid computation

**Polynoms**

- Polynomial IOP (interactive oracle proof)

**Proof system:**

- Polynomial commitments



 License: Apache 2.0

# Arithmetic circuit (SNARK)

Defined over an *F* finite field of a *p* prime

DAG: directed acyclic graphs

Inputs, gates, constants, connections (wires)

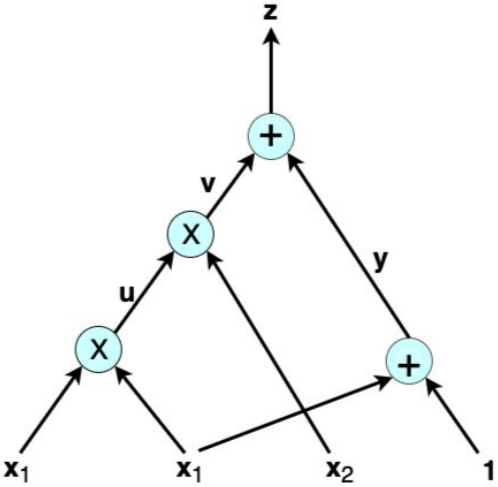
Nodes are labelled with "+" addition and "*x*" multiplication

Inputs are *1, x1, x2, ….. xN*.

Public and private inputs

Defined an n variate polynomial with evaluation recipe

E.g. *C* arithmetic circuit for proving the preimage of a hash function: $C_{SHA}(h, m)$: outputs 0 if $SHA_{256}(m) = h$ , and $\neq 0$ otherwise



 License: Apache 2.0

# Arithmetic Intermediate Representation (STARK)

State of the computation is based on registers taking values from a finite field

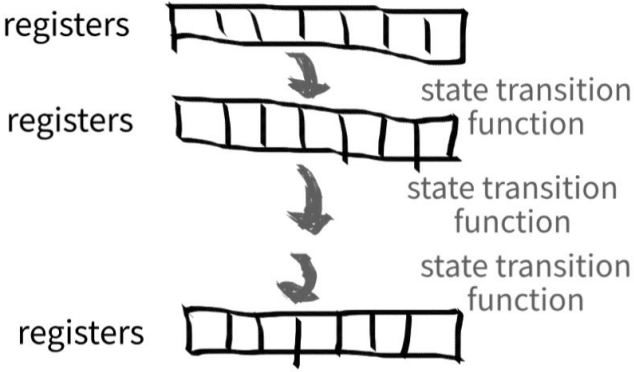
State transaction function

Algebraic execution trace

Boundary constraints

Transition constraints

algebraic intermediate representation (AIR)



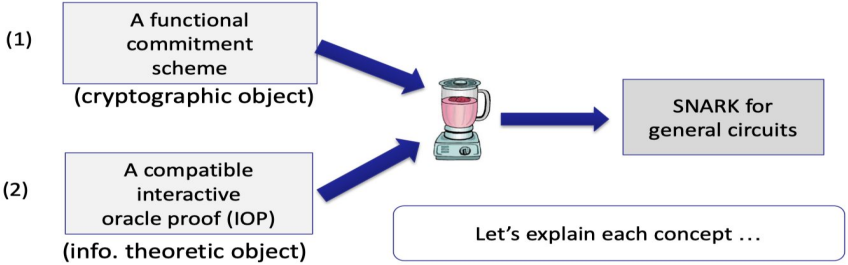
 License: Apache 2.0

# Constructions

Polynomial commitment scheme: get succinct interactive argument: Bind and commit a polynomial.

Interactive Oracle Proof (IOP): R1CS or circuit satisfiability. Evaluating (committed) polynomials in one or multiply points in multiply rounds, extend polynomial commitment to general satisfiability.

Fiat-Shamir: transforming a interactive proofs to non-interactive:



# Zoo of Zero knowledge Proofs

**Construction**

Several polynomial IOP

Several polynomial commitments

Mixing up components:
- time (verification, proof)
- proof size
- setup assumptions
- post quantum

**Polynomial IOP**

Interactive proof:
- vSQL, Libra

Multi-prover interactive proofs:
- Spartan

constant -round polynomial IOPs:
- Marlin, Plonk

**FRI**
**(Fast Reed-Solomon Interactive Oracle Proof of Proximity)**

**Polynomial commitment**

Pairing:
- KZG10

Discrete logarithm:
- Bulletproof

Hashing, merkle tree:
**- FRI:(Fast Reed-Solomon Interactive Oracle Proof of Proximity)**

Groups of unknown order:
- DARK

License: Apache 2.0

# Platforms

## SNARK

Languages / provers:

- Circom
- Noir
- ZoKrates
- Leo
- Halo2

Platforms:

- Aztec
- ZK-Sync
- Aleo

## STARK

Languages / provers:

- Cairo

Platforms:

- StarkNet: L2 rollups
- StarkEX: L2 execution environment

Try right clicking on a photo and using "Replace Image" to show your own photo.

License: Apache 2.0

# Challenge

**Experiment with a STARK system on your own**

License: Apache 2.0

# Links, Resources, Literature

STARK 101
https://starkware.co/stark-101/

Anatomy of a STARK
https://aszepieniec.github.io/stark-anatomy/

Stark-book
https://github.com/zksecurity/stark-book

stark-anatomy
https://github.com/aszepieniec/stark-anatomy

Yet another circle STARK tutorial
https://research.chainsafe.io/blog/circle-starks/

License: Apache 2.0

# *Happy Hunting for the STARK :)*

# *Q & A*

Daniel Szego
In: https://www.linkedin.com/in/daniel-szego/